

# Last Race Code Walkthrough Study Guide

Use this guide to prepare for a line-by-line oral exam. It explains the purpose of each important file, the role of each function, and the answer you can give when the professor asks why something was implemented this way.

## 1. Project Architecture

- The project uses the required two-server SPA architecture.
- client/ is a React 19 + Vite application running on localhost:5173.
- server/ is a Node.js + Express API server running on localhost:3001.
- SQLite stores users, metro network data, events, games, and game steps.
- React never reads SQLite directly. It calls HTTP APIs.
- Authentication uses Passport.js with session cookies.
- The database is pre-populated in server/data/last\_race.sqlite and can be reset with server/init\_db.js.

**What to say: "The client is responsible for UI state and user interaction. The server owns authentication, database access, game setup, route validation, random events, and persistence."**

## 2. Database Design

Tables:

- users: registered users; password\_hash stores bcrypt salted hashes, never plain passwords.
- stations: one row per station, with display coordinates for the map.
- metro\_lines: one row per metro line, with name and color.
- line\_stations: many-to-many ordered join table between lines and stations. It avoids duplicating station data and supports interchanges.
- events: random game events and their coin effects.
- games: one game session with user, start, destination, status, timestamps, and score.
- game\_steps: ordered route execution steps, including selected random event and coins after the step.

**What to say: "The schema is normalized. Stations are stored once, and line\_stations models both line order and interchange stations."**

## 3. server/db.js

Lines 1-4: Import sqlite3 and Node helpers for file paths and directory creation.

Lines 6-10: Build an absolute path to server/data/last\_race.sqlite using import.meta.url. This

is Linux-compatible and does not rely on the current working directory.

Line 12: Creates the data directory if it does not exist.

Line 14: Enables sqlite verbose mode for clearer debugging.

Lines 16-20: openDatabase creates a SQLite connection and turns on foreign key enforcement.

Line 22: Opens the default shared database connection used by the server.

Lines 24-34: run wraps db.run in a Promise and returns lastID and changes. It is used for INSERT, UPDATE, DELETE.

Lines 36-46: get wraps db.get in a Promise. It returns one row, used for users and single games.

Lines 48-58: all wraps db.all in a Promise. It returns multiple rows, used for lists and rankings.

Lines 60-70: exec wraps db.exec in a Promise. It executes multi-statement SQL, mainly schema creation.

Lines 72-82: closeDatabase closes the SQLite connection cleanly.

**What to say: "sqlite3 is callback-based, so I created small Promise helpers to use async/await throughout the server."**

## 4. server/init\_db.js

Lines 1-10: Import bcrypt and database helpers.

Line 12: SALT\_ROUNDS controls bcrypt hashing cost.

Lines 14-31: Define deterministic station seed data, including x/y map coordinates.

Lines 33-54: Define four metro lines and their ordered stations.

Lines 56-65: Define at least eight events with effects between -4 and +4.

Lines 67-71: Define three seeded registered users.

Lines 73-103: Define two completed games for alice and bob.

Lines 105-176: createSchema creates all tables with primary keys, unique constraints, checks, and foreign keys.

Lines 109-114: users table stores username, display\_name, and password\_hash.

Lines 116-121: stations table stores unique station names plus map coordinates.

Lines 123-127: metro\_lines table stores line names and colors.

Lines 129-138: line\_stations table stores each line's station order and connects metro\_lines to stations.

Lines 140-144: events table stores description and effect with a CHECK constraint.

Lines 146-159: games table stores each game session and references users and stations.

Lines 161-174: game\_steps stores the ordered execution path and references games, stations, and events.

Lines 178-265: seedData inserts stations, lines, events, users, and completed games.

Lines 186-218: Insert stations and line\_stations, using Maps to remember generated IDs.

Lines 220-230: Insert events and keep their IDs for game steps.

Lines 232-240: Hash the password "password" with bcrypt before inserting users.

Lines 243-265: Insert completed games and their game\_steps.

Lines 267-281: main closes any old connection, deletes the old SQLite file, recreates schema, seeds data, and prints success.

Lines 283-287: If initialization fails, print the error and exit with code 1.

**What to say: "The seed is deterministic because all data is defined in arrays. The script**

safely recreates the database file, so repeated runs give the same initial state."

## 5. server/game\_logic.js

Lines 1-4: makeSegmentId creates a stable id for an undirected segment by sorting station IDs. A-B and B-A produce the same id.

Lines 6-36: buildSegments reads ordered line stations and creates unique station-to-station segments. It also tracks which line or lines serve each segment.

Lines 38-54: buildAdjacency creates a graph representation where each station points to neighboring stations.

Lines 56-76: shortestStopDistance runs breadth-first search to compute the minimum number of stops between two stations.

Lines 78-85: findCandidateDestinations keeps only stations reachable at least minimumStops away from the start.

Lines 87-149: validateRoute checks the submitted route.

Lines 92-94: Empty route is invalid.

Lines 96-116: Each selected segment must exist and connect to the current station.

Lines 118-120: Final station must match the assigned destination.

Lines 122-146: Check line-change rules. A line change is valid only if it happens at an interchange station.

Line 148: Return valid route and normalized steps for persistence.

**What to say: "The game is a graph problem. I use BFS for distance and a separate validator for the submitted route rules."**

## 6. server/index.js

Lines 1-14: Import Express, CORS, sessions, Passport, bcrypt, database helpers, and game logic helpers.

Lines 16-20: Configure app constants: port, allowed origins, initial coins, and minimum distance.

Lines 22-33: express.json parses request bodies; CORS allows the Vite client and includes credentials.

Lines 35-46: express-session creates cookie-based sessions.

Lines 47-48: Initialize Passport and attach session support.

Lines 50-70: LocalStrategy authenticates username/password. It fetches the user, compares bcrypt hash, and returns a public user.

Lines 72-74: serializeUser stores only user.id in the session.

Lines 76-83: deserializeUser reloads the public user from SQLite on later requests.

Lines 85-91: toPublicUser removes password\_hash before sending user data to the client.

Lines 93-100: ensureAuthenticated protects private APIs.

Lines 102-106: asyncHandler forwards async errors to Express error middleware.

Lines 108-151: loadNetwork queries stations, lines, line\_stations, builds segments, and finds interchange station IDs.

Lines 153-155: selectRandomItem chooses one item from an array.

Lines 157-163: root and health endpoints prove the server is running.

Lines 165-178: Public instructions endpoint. Anonymous users can read rules.  
Lines 180-200: POST /api/sessions logs in and creates a session.  
Lines 202-204: GET /api/sessions/current returns current user or null.  
Lines 206-221: DELETE /api/sessions/current logs out and destroys the session.  
Lines 223-234: GET /api/network is protected and returns stations, lines, and segments.  
Lines 236-284: POST /api/games creates a new in-progress game with random start and destination at least 3 stops apart.  
Lines 286-386: POST /api/games/:id/route validates and executes a submitted route.  
Lines 310-325: Invalid route becomes completed game with score 0.  
Lines 329-354: Valid route applies one random event per step and inserts game\_steps.  
Lines 356-370: finalScore is max(0, coins), then games is updated as completed.  
Lines 388-413: GET /api/games/history returns the logged-in user's games.  
Lines 415-436: GET /api/ranking returns each user's best completed score.  
Lines 438-441: Central error handler returns JSON instead of crashing.  
Lines 443-445: Start listening on port 3001.

**What to say: "The client never decides if a route is valid. It submits segment IDs; the server validates and persists the authoritative result."**

## 7. client/src/api.js

Line 1: API\_BASE points React to the Express API server.  
Lines 3-24: request is a shared fetch wrapper.  
Line 5: credentials: "include" sends and receives the session cookie.  
Lines 6-10: Adds JSON headers and merges any options from callers.  
Lines 13-15: 204 means no JSON body, used for logout.  
Lines 17-23: Parse JSON, throw Error on non-OK responses, otherwise return payload.  
Lines 26-44: api object exposes readable methods for session, network, game, ranking, and history calls.

**What to say: "The fetch wrapper centralizes cookie handling and error handling so components stay simpler."**

## 8. client/src/App.jsx

Lines 1-19: Import React hooks, router tools, lucide icons, API helper, and CSS.  
Lines 21-25: AuthContext and useAuth share logged-in user and auth actions across components.  
Lines 27-92: App component stores session state, checks current session on load, defines login/logout methods, and declares all routes.  
Lines 94-154: PageShell renders the top bar, nav links, login/logout area, and page content.  
Lines 156-159: ProtectedRoute redirects anonymous users to /login.  
Lines 161-182: CharacterScene renders decorative CSS-only train, coin, signal, sparkles. It has aria-hidden because it is not functional content.  
Lines 184-226: HomePage fetches public instructions and shows the anonymous rules page.  
Lines 228-276: LoginPage handles username/password form, calls api.login, stores errors, and navigates to /play after success.

Lines 278-495: PlayPage manages the game flow.  
Lines 279-287: State variables: network, gameData, phase, selectedSegments, timer, result, visible step, error, and submittingRef.  
Lines 289-309: submitRoute sends selected segment IDs to the server and updates result/phase.  
Lines 311-313: On mount, fetch network data.  
Lines 315-325: Timer effect counts down only during planning.  
Lines 327-334: When timer reaches zero, auto-submit the current route.  
Lines 336-339: stationById memo builds a lookup Map for station labels.  
Lines 341-355: currentStationId is derived from selected segments.  
Lines 357-368: startGame calls the server and resets local game UI state.  
Lines 370-376: addSegment appends a clicked segment during planning.  
Lines 378-389: showNextStep reveals execution events one at a time.  
Lines 394-396: Derived values for planning view.  
Lines 398-493: Render map panel and side panel depending on phase.  
Lines 497-564: NetworkMap draws the SVG map, full colored lines in setup, selected route during planning.  
Lines 566-635: RouteBuilder sorts valid next segments to the top, supports undo/clear/submit.  
Lines 637-665: ExecutionView shows route events step by step.  
Lines 667-688: ResultView shows valid/invalid route result and link to ranking.  
Lines 690-723: RankingPage fetches and renders general ranking.  
Lines 725-762: HistoryPage fetches and renders the logged-in user's game history.  
Line 764: Export App.

**What to say: "React controls visual phases and selected route state, but server APIs are the source of truth for game creation, validation, random events, and saved scores."**

## 9. client/src/App.css and index.css

App.css defines visual layout, topbar, cards, map, tables, buttons, route list, and decorative animations.

Important ideas:

- .app-shell creates the page background.
- .topbar styles navigation.
- .play-grid lays out map and side panel.
- .network-map styles the SVG map.
- .segment-list styles the selectable route buttons.
- .character-scene, .train-character, .coin-character, .signal-character define decorative CSS-only characters.
- @media (prefers-reduced-motion: reduce) disables animations for accessibility.

index.css sets base font, box sizing, body margin, and minimum desktop width.

**What to say: "The CSS is presentation only. It does not contain game logic."**

## 10. Common Oral Exam Questions

**Q: Why use line\_stations?**

A: Because a station can belong to multiple lines. The join table avoids duplicate station rows and stores station order per line.

**Q: Where is the password stored?**

A: In users.password\_hash as a bcrypt salted hash. Plain passwords are never stored.

**Q: How is login remembered?**

A: Passport serializes user.id into the session cookie. On each request, deserializeUser reloads user data from SQLite.

**Q: Why credentials: "include" in fetch?**

A: The client and server are different origins, so cookies are sent only if fetch includes credentials and server CORS allows credentials.

**Q: How do you ensure destination is at least 3 stops away?**

A: The server builds graph adjacency and uses BFS shortestStopDistance before choosing a candidate destination.

**Q: How is a route validated?**

A: Server checks that every segment exists, connects to current station, reaches destination, and changes lines only at interchange stations.

**Q: Why does the server choose random events?**

A: Randomness and score calculation must be server-side so the client cannot cheat.

**Q: What is stored after a game?**

A: games stores summary and final score; game\_steps stores each executed step, event, and coin total.

**Q: What happens for invalid routes?**

A: The game is marked completed with final\_score 0 and no game\_steps are inserted.

**Q: Why use React state and hooks?**

A: State represents UI data; effects synchronize with APIs and timer; memo avoids recalculating station lookups unnecessarily.

## 11. How To Study

1. First memorize the architecture: React -> API -> SQLite.
2. Then learn the database tables and why line\_stations exists.
3. Then study server/index.js routes because they are the application behavior.
4. Then study game\_logic.js because it is the rules engine.
5. Then study App.jsx by components: App, PageShell, LoginPage, PlayPage, NetworkMap,

RouteBuilder.

6. Do not memorize CSS line by line. Know which classes control layout, map, buttons, and animations.